QCon
全球软件开发大会

InfoQ 技术大会

OpenCodeReview：百万真实任务验证的确定性工程与 Agent 协同

李峥峰

阿里巴巴 高级研发工程师

关于我

Open Code Review 作者、阿里集团 AI 代码评审负责人。主要研究 AI Agent、程序分析等方向，先后负责了集团内代码导航、代码分析、代码评审智能化等项目的落地，服务了阿里集团内的数万开发者，在实际业务中取得显著效果。在 AI Agent 系统设计、场景化工具链蒸馏、大规模生产环境下的质量 - 成本 - 速度平衡等方面积累了一定的经验。开源了 Open Code Review 项目，开源数月内多次登顶 GitHub 日榜、周榜第一，npm 社区的包下载量超过百万。

BUILDING IN THE OPEN

关注我正在做的事

开源项目、工程实践与模型探索，欢迎一起交流。

- **Open Code Review**
  AI 代码评审工具以及 benchmark 的持续迭代
- **AI Agent 工程实践**
  确定性工程、工具链与真实场景验证
- **代码评审领域大模型的探索**
  质量、成本与速度的更优平衡

GitHub

github.com/lizhengfeng101

扫码访问主页 · 关注后续进展

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 瓶颈没有消失而是发生了转移

02 阿里内部长达两年的大规模生产实践

03 什么情况下需要打造垂直领域的 Agent?

04 OpenCodeReview：确定性工程 x Agent 协同

05 如何验证：从用户行为指标到客观评估

06 OpenCodeReview 自身的 RSI 实践

07 思考与展望

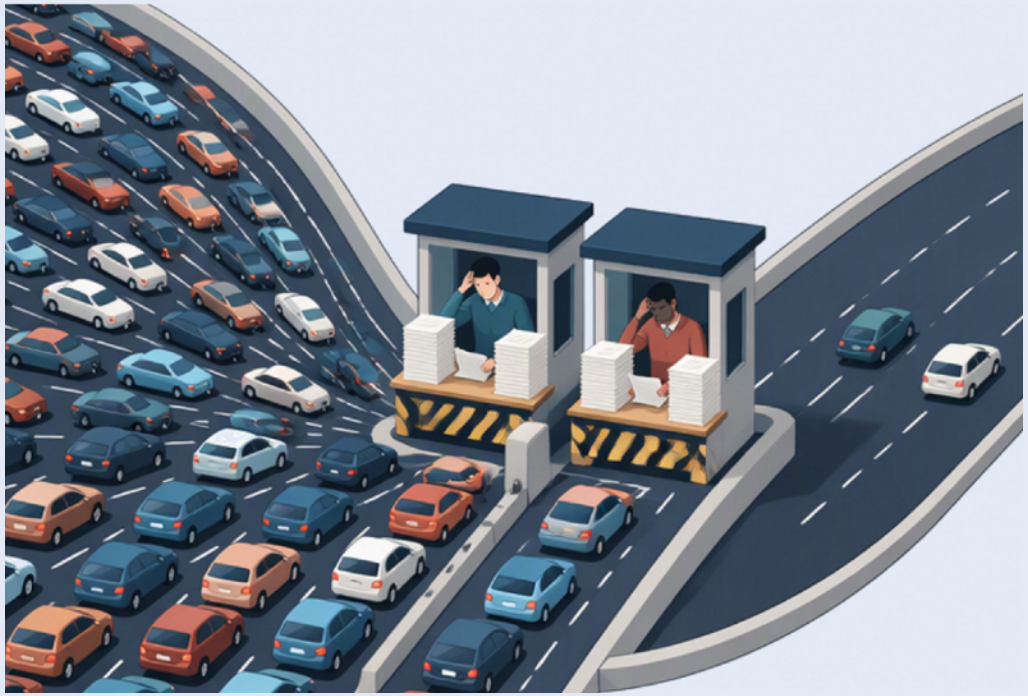
01

瓶颈没有消失而是发生了转移

写代码快 10 倍，交付也会快 10 倍吗?

答案是：否定的!

AI 时代下不同资产类型的效率瓶颈有所不同



重要资产：AI 加速了代码生成，但是写得快，不等于交付快

非重要资产：前期很快，理解与维护的代价推给了未来，越改越难懂，修一处坏一处，无人敢发。

非长期迭代的项目不存在这些问题（尤其面向非研发），提效最大

吞吐量上去之后，质量发生了什么变化?

Faros AI 在26年4月发布了 《The Acceleration Whiplash》报告，基于约 22,000 名开发者、4,000+ 个团队、跨越两年的工程遥测（telemetry）数据，使用的是实际系统数据而非开发者自我报告的估算值。

身边也有越来越多的人跟我说同一个问题：代码是 AI 写的，看不过来，不敢合

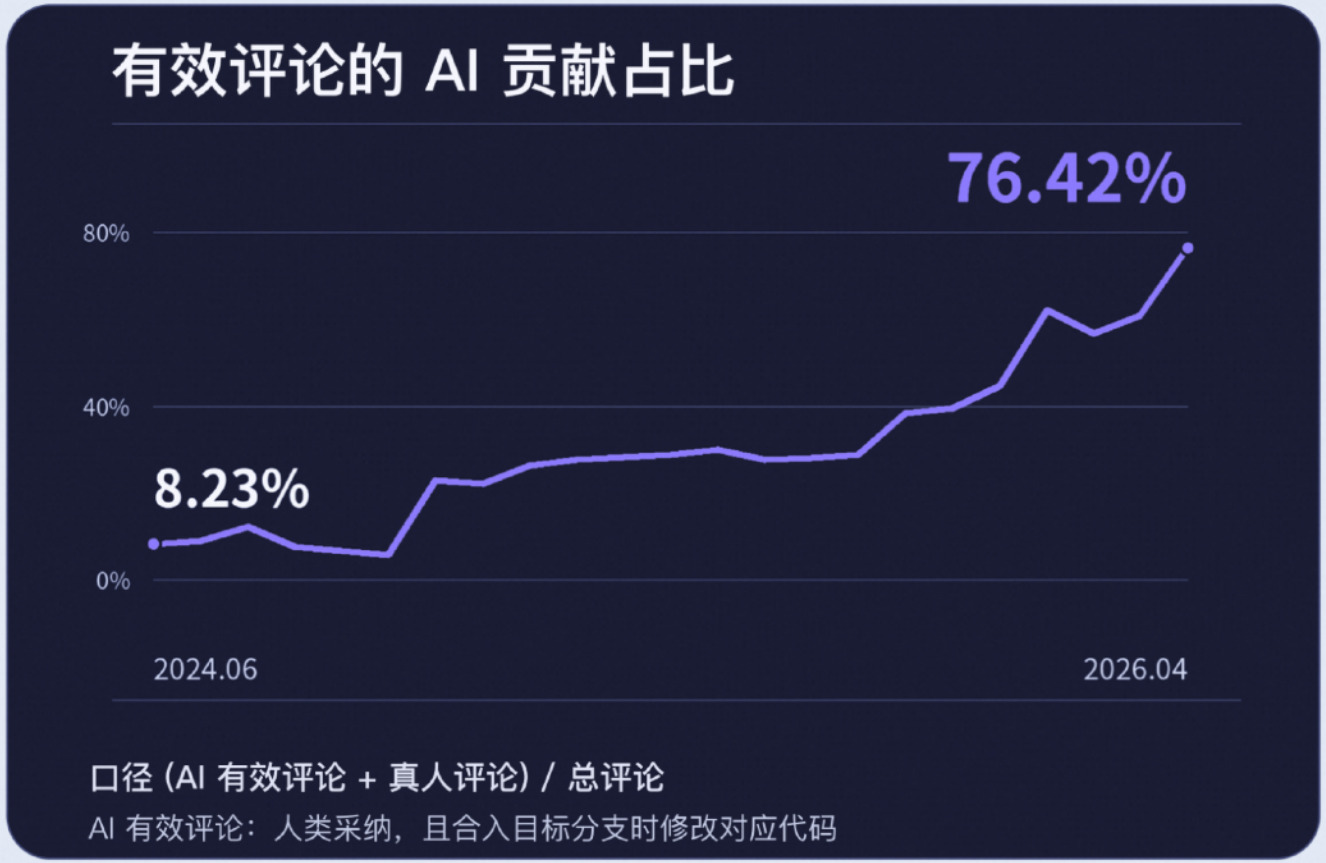
02

阿里内部长达两年的大规模生产实践

阿里内部：百万级任务的大规模生产验证

过去的两年时间，AI 正在将人类从繁琐的评审工作中解放出来，随着模型和工程能力提升，信任与验证的比例一直在变化。AI 正在接管代码层面的评审，人类聚焦于处理变更过程中的复杂问题。

- OpenCodeReview 成为阿里内部数万工程师日常使用的 AI 代码评审工具。
- 运行了数百万的真实代码评审任务、拦截了数百万个真实代码缺陷。
- 总体评审建议同比翻倍，其中人类提出的评审建议逐渐减少，人类更专注于审查复杂的业务问题。
- 绝大部分的历史编码故障都能被识别。



开源社区：全球开发者的广泛传播与认可

7 天时间从 0 使用 Go 重写，自 26年6月正式开源后，迅速登上 HackNews 头条，多次登顶 Github Trending 日榜、并且周榜和 Go 语言月榜获取第一，成为 Alibaba GitHub Org 中 Star 数 No.1 的开源项目，社区自发分享上千篇。

Secure, fast, efficient

- **确定性工程 + Agent**
  工程处理边界，Agent 专注语义推理
- **精准定位与反思**
  行级评论定位，反思拦截幻觉与偏移
- **多模型协议支持**
  兼容主流模型 API，支持私有端点
- **开箱即用的生态集成**
  支持 Skills、IDE 插件、GitHub / GitLab CI 等
- **多轮递进审查**
  Effort 控制深度，无新增风险时收敛
- **内置审查规则**
  覆盖 40+ 种语言与文件类型

GITHUB TRENDING
#1 Repository Of The Day

TRENDSHIFT
#1 Repository Of The Week

TRENDSHIFT · Go
#1 Repository Of The Month

47K+
GitHub Stars

100万+
npm 下载量

200+
全球贡献者

OpenSSF Gold
开源最佳实践

扫码访问开源项目
github.com/alibaba/open-code-review

03

什么情况下需要打造垂直领域的 Agent?

三个代表性的时代：CoT、Workflow 与 Agent 代表不同时代的实践重心

2023年

复杂问题拆解、多步推理

策略固定、强依赖模型自身

2024年

预定义流程编排、集成多模型、多工具，

例如固定加载 n 层调用链，提供文件语法结构等

流程固定、灵活性差、维护成本极高、上下文模式偏固态

2025年-至今

“感知→规划→行动→观察”自主闭环、动态决策，通过工具与真实世界实时反馈

难以调试、token 消耗极大、自主探索带来覆盖、权限、成本与审计挑战

AI 报告已经生成，评审真的完成了吗？

步入 Agent 时代，我们使用大量的真实 PR 回归通用 Agent，发现存在一些问题，这些问题在大规模生产环境放大后，是难以解决、难以接受的。

一次 PR 的评审过程 机制示意

✓	文件 A	已阅读
!	文件 B	漏审
...	大文件 C	只读局部
✓	文件 D	已阅读

Agent 的输出

评审报告已生成

包含结论与评论建议

报告不一定等于完整的评审

01 覆盖不完整

漏文件、长文件处理不完整

02 定位漂移

评论位置与实际代码偏移

03 执行失控

过度调用工具、bash 权限过大

04 规则难维护

规则堆进大 agent.md，难以按文件匹配

把脚本写进 skills，再把 Skills 做成了垂直 Agent

我们尝试使用大量的脚本来收敛 Agent 的行为，这些脚本确实缓解了通用 Agent 的一些问题，也收敛了效果；但是规模化之后，维护与执行治理成了新的问题。

把确定性的事，交给脚本

脚本化 Skills

- **覆盖盲区** → 解析 diff，逐文件维护待审清单
- **定位漂移** → 滑动窗口匹配，校准评论行号
- **执行失控** → 封装工具调用，收紧执行权限
- **规则难维护** → 拆分规则文件，按目标文件注入

→

有效，但越来越难治理

规模化代价

- **维护成本攀升**
  规则、脚本、提示词彼此耦合
- **运行状态难追踪**
  漏审、重试、超限难以统一观测
- **算力调度难统一**
  大规模并发下需要明确的任务状态

脚本解决单点问题，却不能替代全局执行治理。

转折：保留脚本的确定性，把执行框架收回自己手里。

自建评审 Agent，是为了把覆盖、权限、重试与观测变成系统能力。

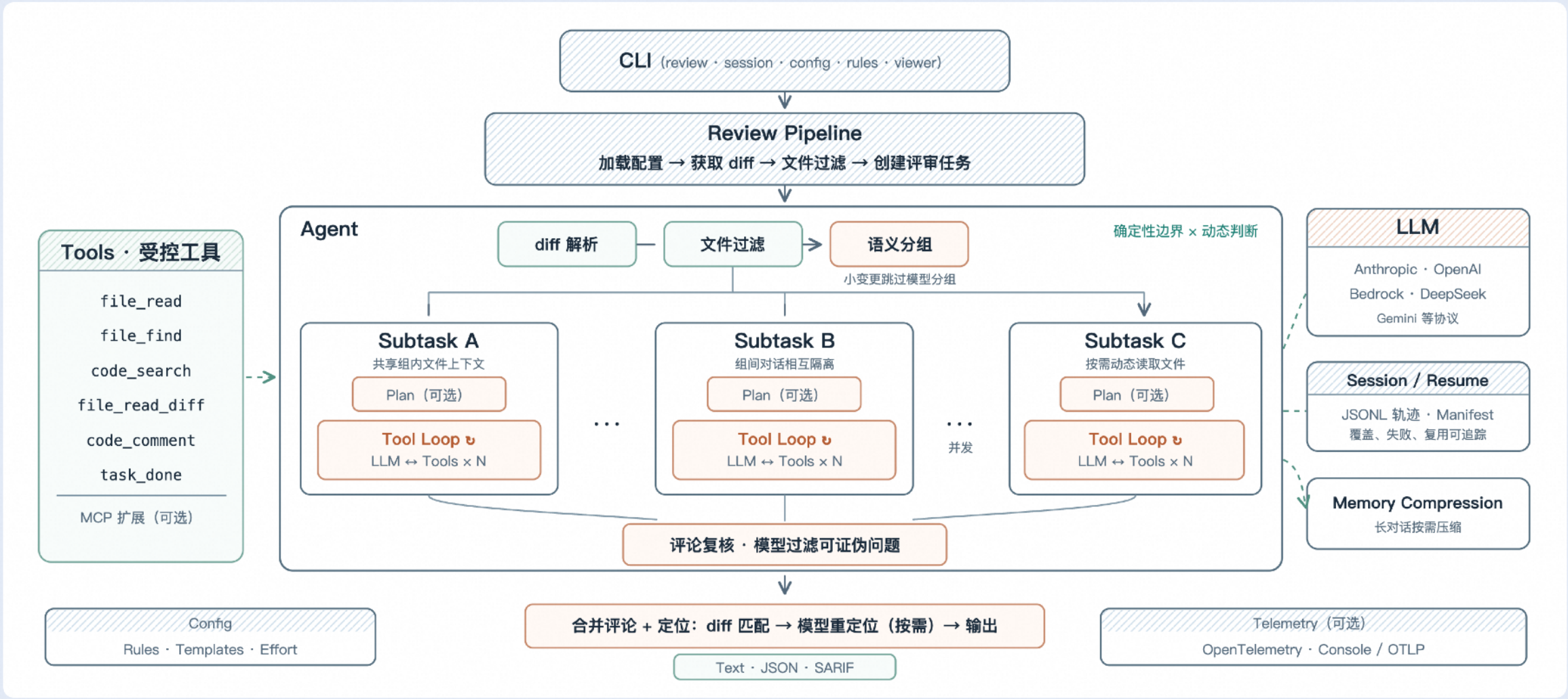
脚本化 Skills → 自建 Agent

04

OpenCodeReview：确定性工程 x Agent 协同

边界交给工程，决策留给 Agent

从通用 Agent 的大量评审轨迹中收敛提示词、工具与流程：确定性问题交给工程，动态决策交给 Agent。



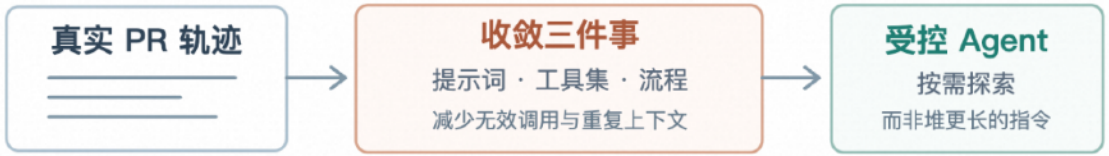
Less is More：让 Agent 专注做它擅长的事

四个机制相互配合，让 Agent 执行更专业、更安全、评审效果更稳定、响应与 token 成本更可控。

01 提示词、工具、流程收敛

从轨迹到产品

从大量通用 Agent 评审轨迹，找出有效动作和浪费。



把反复出现的操作做成系统，把未知问题留给 Agent。

02 文件筛选与分组

边界确定 · 上下文组织

先筛掉无须评审的文件，再让相关文件一起被理解。

工程确定评审范围，模型进行语义分组。

03 规则引擎

按文件动态加载

不同文件，不必背同一本“厚规则书”。

工程决定规则如何匹配，Agent 决定是否存在违规问题。

04 评论反思与定位

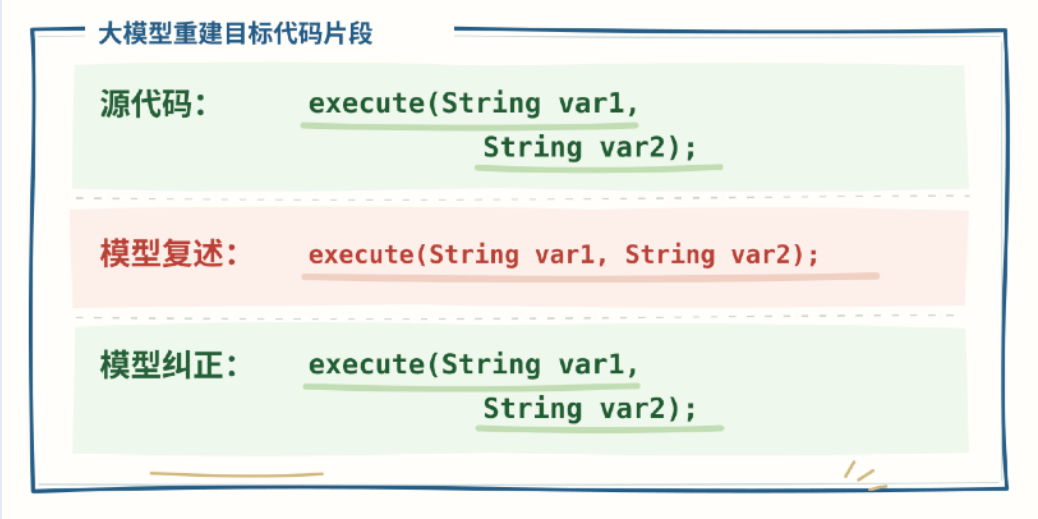
独立的结果处理

发现问题之后，还要验证、找到它在代码里的位置。

模型负责可疑问题的判断，工程负责校准与标明定位失败。

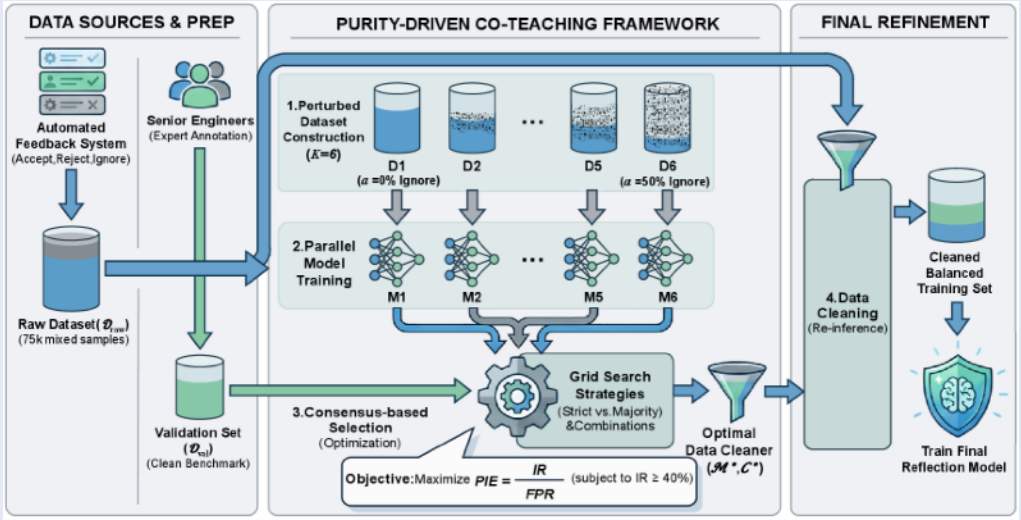
专项模型的尝试：把高频子任务做准、做快

针对评论定位与误报过滤两类高频、边界清晰的子任务，利用真实业务数据微调专项模型，在提升任务效果的同时降低处理时延。



定位模型：Qwen3-8B

利用线上真实数据（定位成功、定位失败、使用 SOTA 模型纠正失败）进行训练，用于传统算法定位失败时使用大模型重建目标代码片段。该任务训练后，相比基模的成功率从 37.35% 提升到 85.65%，平均耗时从大尺寸主模型的 3s 降低到 1s 内。



反思模型：Qwen3-30B-A3B

利用用户反馈收集带标注（采纳、误报、忽略）的数据，训练一个反思模型作为过滤器来拦截误报。针对生产环境标注噪声高、清洗成本大的挑战，我们以少量专家及用户反馈数据为锚点，通过多模型协同标注从噪声中提取可靠样本，训练反思模型拦截误报。最终有效过滤了大模型幻觉，误报拦截率从30.09%提升至52.63%，耗时由5s降至500ms内。

一次评审不只有报告，还有可追踪、审计的 Manifest

人们越来越依赖 AI 评审报告，也越来越多地让另一个 Agent 应对评审报告。报告反应了评审过程发现了哪些问题，Manifest 则交代哪些文件审完、哪些未完成以及原因，让人和下游 Agent 都能核对评审范围，才能更好地信任、复查与修复。

只有输出

评审报告已生成

缺陷 中

pom.xml

新增依赖版本包含 `-SNAPSHOT`，不符合已发布版本要求。请改为可从仓库解析的正式版本。

缺陷 高

src/order/repository.go

数据库连接初始化失败时，`r.db` 可能为 nil；直接调用 `QueryContext` 会触发空指针异常。请先检查连接是否可用。

却看不出哪些文件未完成。

不只看评论数

Run Manifest

PARTIAL / 部分完成

每个应审文件，都有执行结果

选中评审 · 4 个文件			完成 2 · 复用 1 · 失败 1
01	pom.xml	已完成	本轮执行
02	src/order/repository.go	已完成	本轮执行
03	src/order/config.go	已复用	沿用已有结果
04	src/order/handler.go	未完成	预算耗尽 · 待处理

筛选结果 · 不计入 Manifest 的应审分母

–	vendor/dependency.go	已过滤	目录规则排除

逐文件状态 + 失败原因 + 运行终态

轨迹可追溯 →

05

如何验证：从用户行为指标到客观评估

当 AI 替人处理评论，“采纳”还能证明什么？

用户行为曾经是最重要的反馈，但随着 AI 的参与度越来越高，研发流程发生了重大的改变，人类的行为指标与客观的缺陷真值正在分离。

参与方式变了，行为指标的含义也会变

随着 AI 的发展，人的行为也在发生变化。

人工逐条看

阅读代码、讨论建议、手动修改

行为较接近判断

→

AI 辅助筛选

人快速确认，由工具帮助理解

行为掺入效率选择

→

Agent 接手

读评论、改代码、决定采纳或忽略

行为未必代表人的判断

同一个“采纳”动作，背后可能是理解、信任，或另一个 Agent 的一次概率性的选择。

这些指标都仍有用，但权重在降低

不能用单一行为代表评审本身的能力。

采纳率

建议被处理，不代表问题有价值

忽略率

未处理，不代表评论无效

误报反馈

反哺 Agent 迭代的重要信息来源

过时率

代码合入前是否被修改的客观标准

最无法靠“采纳”回答的：真实问题里，还有多少没有被发现？

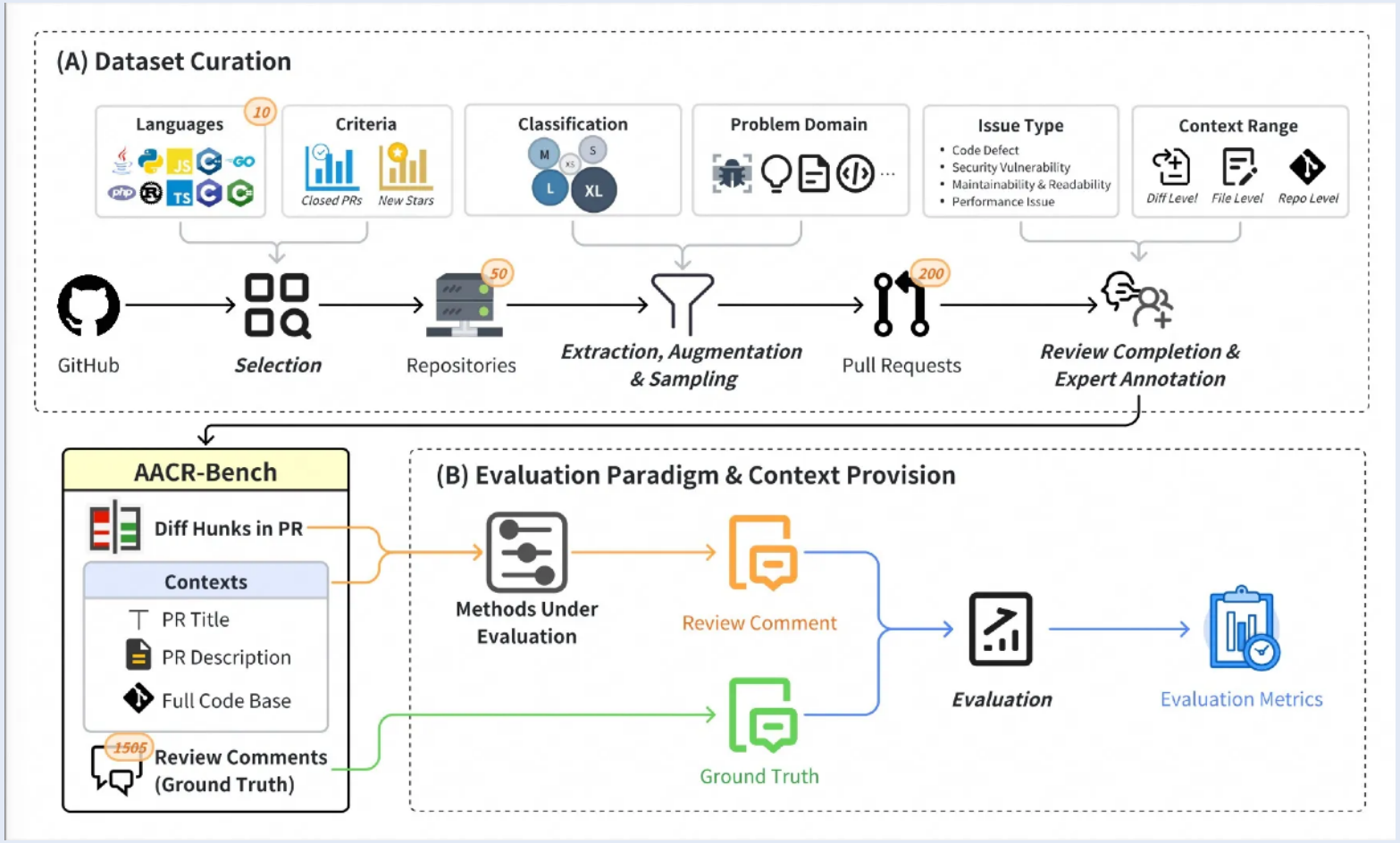
用户行为不再是缺陷真值

AACR-Bench：客观量化评审任务的效果

2026 年初，我们开源了首个多语言、仓库上下文感知的 CodeReview 评测集。

人机结合：采用“AI 辅助 + 人类专家校验”的标注流水线，80 多位资深工程师交叉标注。相比传统数据集，问题覆盖率提升了285%，挖掘出大量原 PR 中被忽略的缺陷。

多维度评估：更全面的评估维度打破语言壁垒，支持10 种主流编程语言，并提供完整的仓库级依赖上下文，覆盖了多种类型与多种作用域的代码问题。这使得评估场景能够真实还原复杂的跨文件代码审查过程，全面考验模型的系统性理解能力。



模型相同，不同的评审工具有哪些差异?

OpenCodeReview

通过 CLI 发起评审

```
> ocr review
```

主模型　GPT-5.5 / Claude-4.6-Opus

Codex

使用 /review 评审入口

```
> /review
```

主模型　GPT-5.5

Claude Code

使用 /code-review 评审命令

```
> /code-review
```

主模型　Claude-4.6-Opus

排名	模型	来源	F1	精确率	召回率	平均耗时	平均 Token
1	Claude-4.6-Opus Anthropic	Open Code Review v1.3.1	25.10%	33.90% 301/889	20.00% 301/1505	1m23s	385K 375K / 10K
2	GPT-5.5 OpenAI	Open Code Review v1.3.1	21.00%	32.10% 234/728	15.50% 234/1505	2m51s	422K 409K / 13K
3	GPT-5.5 OpenAI	Codex v0.140.0	8.36%	27.82% 74/266	4.92% 74/1505	2m58s	525K 520K / 5K
4	Claude-4.6-Opus Anthropic	Claude Code v2.1.169	11.57%	7.23% 435/5980	28.90% 435/1505	13m6s	5,664K 5,603K / 60K

OpenCodeReview 在不同模型下的表现

排名	模型	来源	F1	精确率	召回率	平均耗时	平均 Token
1	Claude-4.8-Opus Anthropic	Open Code Review v1.3.1	17.90%	37.80% 176/465	11.70% 176/1505	1m6s	352K 342K / 11K
2	Claude-4.6-Opus Anthropic	Open Code Review v1.3.1	25.10%	33.90% 301/889	20.00% 301/1505	1m23s	385K 375K / 10K
3	Qwen3.8-Max Alibaba	Open Code Review v1.8.7	23.00%	33.90% 262/774	17.40% 262/1505	5m14s	334K 303K / 32K
4	GLM-5.2 Zhipu AI	Open Code Review v1.3.1	21.30%	32.30% 239/741	15.90% 239/1505	7m58s	682K 624K / 58K
5	GPT-5.5 OpenAI	Open Code Review v1.3.1	21.00%	32.10% 234/728	15.50% 234/1505	2m51s	422K 409K / 13K
6	Deepseek-V4-Pro DeepSeek	Open Code Review v1.3.1	17.90%	30.60% 191/624	12.70% 191/1505	6m28s	394K 350K / 44K
7	GLM-5.1 Zhipu AI	Open Code Review v1.3.1	20.40%	28.90% 237/820	15.70% 237/1505	4m11s	743K 707K / 36K
8	Qwen3.7-Max Alibaba	Open Code Review v1.3.1	21.20%	25.20% 276/1096	18.30% 276/1505	4m41s	625K 587K / 38K

Benchmark 驱动迭代：衡量效果、定位原因

评审质量不能只看用户采纳。用固定样本与缺陷真值衡量结果，结合执行轨迹定位原因，再通过回归验证改进。

01 / PROCESS · 过程分析

Agent 如何完成评审?

在固定输入下核对每个环节，而不是只看最终评论。

文件覆盖
完成、失败、未审文件

分组情况
组大小、成员与失声文件

工具调用
成功率、提交时点与失败

执行预算
轮次、耗时与 Token

02 / OUTCOME · 结果分析

真实问题找对、找全了吗?

与人工核验的 Ground Truth 对比，不能只凭产出量。

Precision
报出的问题中
有多少是真的?

Recall
已知真实问题中
找到了多少?

F1
准确率与召回率
如何综合取舍?

CASE / 以“语义分组回归”的特性为例 **评论变少，是降噪还是漏报?**

历史版本 · 同批 194 个 PR

评论	Precision	Recall	F1
819 → 493	33.9% → 38.7%	20.0% → 12.7%	25.1% → 19.1%

① 结果报警
准确率上升，但真实命中少了 110 条；不是只过滤了噪声。

② 过程归因
头文件失声；组内评论常攒到最后提交，共享轮次使中断波及整组。

③ 调整并回归
将分组保持可控；针对逐文件覆盖与提交时点设计修复，再用同批 PR 验证。

优化不是盲调提示词：先用结果发现退化，再用轨迹定位机制，最后用可证伪的回归确认收益。

06

OpenCodeReview 自身的 RSI 实践

热度的背后是前所未有的迭代速度

一个备受关注的开源项目，几乎每天都在接收变更、验证变更、发布变更。在这个过程中，我们探索了激进的 All in AI 研发流程。

开源以来

4 个月

从项目发布到持续演进

代码变更

800+

Commits

社区迭代

1,500+

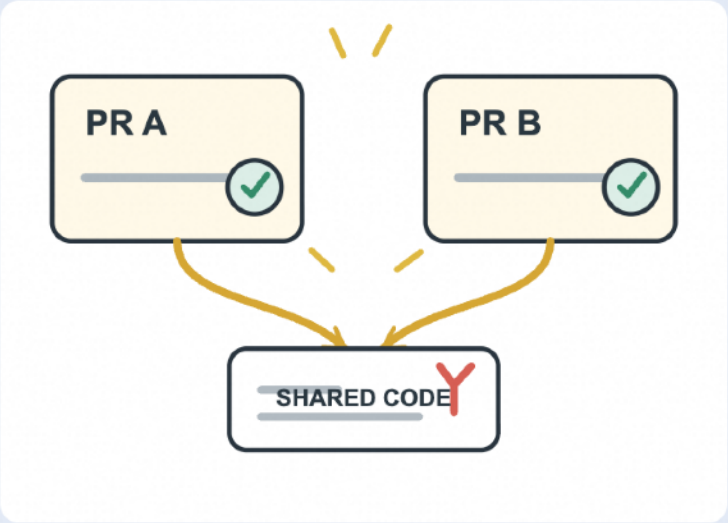
处理的 PR + Issue

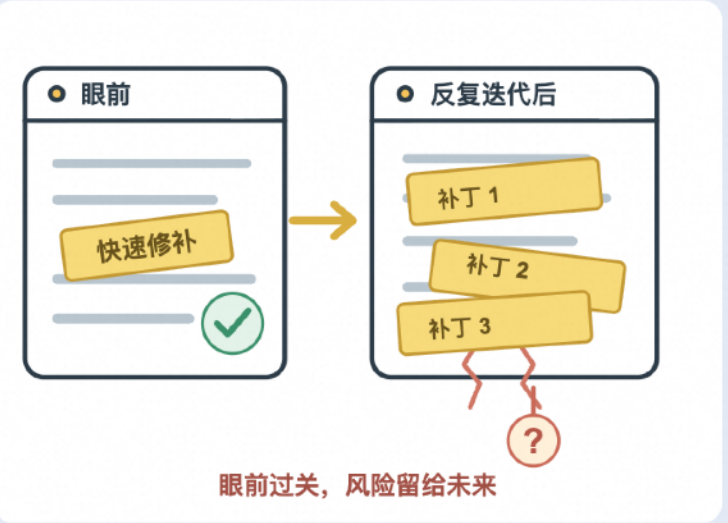
正式交付

140+

正式版本发布

无人值守的自进化（RSI）存在哪些问题?





实现正确 ≠ 需求正确

测试和 AI 评审能验证实现的正确性，却难以判断需求的正确性。看似正确的改动，可能一开始需求就是不成立的。

一起做，还是分开做?

多个 PR 各自评审通过，合在一起却可能功能性冲突。如果推进，需要大局观判断，AI 往往缺乏这些。

Codex: 对，我刚才判断偏了

风险堆积到未来

AI 倾向最快的方式实现需求、解决问题，缺乏大局观；每次看似过关，却把隐患持续堆积到了未来。

人类介入的自进化（RSI）流程

给大模型提供足够的结果、过程轨迹、归因脚本等，由人类专家在关键决策节点介入审批，最终提升整体的迭代效率与稳定性。

合入前质量把关：从自动筛查到人工决策

分层检查变更质量，识别对评审 Agent 的能力影响；人工结合各层结论做出合入决策，任何一层出现问题都要打回重来。

质量漏斗

检查重点与决策依据

规范与一致性 · 静态分析与安全
测试 / Race / 覆盖率 ⩾90% · 跨平台 / 多端构建
契约 / 冒烟验证 · 供应链约束

多工具 · 多模型 · 多规则

交叉排查代码中的逻辑、边界与安全漏洞。

系统设计 · 业务需求 · 影响面

追问需求是否成立，分析跨模块、跨 PR 的长期影响。

是否影响 OpenCodeReview 的评审能力?

评审策略 / 上下文 / 工具 → 建议全量 AACR-Bench
非核心链路改动 → 通常无需全量（如官网样式、README）

综合各层结论，作出最终合入决策

核验关键证据，确认风险与取舍，对合入负责。

07

思考与展望

代码评审工具，不必是单选题

1. 不替代编码 Agent，专注做好评审

很多用户习惯让一个 Agent 写代码、另一个 Agent 评审，避免自写自评。OpenCodeReview 正是这样的角色：OpenCodeReview 专注于代码评审，为已有开发流程补充检查与验证。

2. 评审工具不是单选，多方发现相互补充

在评审环节，用户往往会同时选择多个 CR 工具。工具之间可以互补，而不只是相互替代。

3. 开源热度，折射出更强的验证需求

AI 生成越多代码，开发者越需要多方验证，缓解“能否放心合入”的焦虑。

面向 AI 自主开发时代的下一代评审模式探索

AI 正从“辅助”走向“自主”，人的参与越来越少，但每次合入代码依旧如履薄冰，未来 AI 代码评审会往什么方向演进？从发现代码问题，走向为变更决策提供依据。

过去 · AI 辅助人

检查实现中的缺陷

基于仓库上下文，检查实现是否符合既定需求与方案。

空指针 · 性能 · 业务逻辑

人把关 需求、方案与系统级判断

现在 · 转折点

未来 · 从发现缺陷，到评估需求、设计与演进风险

当下能交付 ≠ 长期可维护

代码暂时没 bug、编译通过、单测通过，代码库却越来越复杂、难以维护。

需求是否合理?

问题是否值得解决，复杂度是否值得

设计与架构是否合适?

职责划分、模块边界与依赖关系

后续是否好改?

扩展、维护与迁移成本

风险是否可接受?

变更影响面与长期代价

以独立证据与取舍分析支撑报告

完整评审报告

问题与影响 · 交付依据 · 风险与取舍

→

人结合经验判断 / 审批

裁决需求歧义、风险与设计取舍

→

完成代码变更

满足条件后合入，否则回修

评测的下一步：拉开差距，对齐体验，持续演进

大模型和 Agent 都在持续进步，评测也需要保鲜，我们后续的方向是基本对齐用户使用体验的同时，扩充数据集，挖掘更高价值的难题。

01 持续保持区分度

已饱和题目降权

主流模型普遍能答对的简单问题，不再主导评分。

补充高价值难题

增加需要深入推理的真实缺陷，体现模型间的能力差异。

02 让结果和体验基本对齐

让评测反映真实优势

真实评审中被用户持续认可的模型，其优势应在评测中体现。

排名反常，必须解释

领先模型若明显落后于小模型或旧模型，需追查与体验背离的原因。

03 构建自迭代闭环

自动挖掘，验证后入集

从历史 PR、修复与反馈中挖掘新问题，验证后持续更新。

人工聚焦争议与校准

未命中标注不直接判错，让人专注争议裁决与抽样校准。

持续更新 候选挖掘 → 问题验证 → 样本生成 → 质量筛选 → 评测与反馈 ↺

从垂直 Agent，走向评审专用模型

垂直 Agent 的价值已在生产中得到验证；下一步，让模型与评审任务、工具和流程共同优化。在评审效果、推理时延与 token 成本之间，寻找更好的平衡。目前已经和 Qwen 基模团队达成合作，一起探索 CodeReview 场景专用的高性能模型。

从专项子任务，拓展到完整评审任务

定位模型

Qwen3-8B

算法定位失败时的模型兜底

反思模型

Qwen3-30B-A3B

复核评审评论，过滤误报

↓ 推广真实业务数据与专项训练经验

探索 CodeReview 专用模型

面向上下文理解、缺陷推理与证据验证，
与 Agent 的工具使用和执行流程协同优化。

不只追求更高分，更要可规模化使用

01 评审效果

发现真实、高价值缺陷，减少误报，让结论有据、建议可执行。

02 推理时延

减少无效探索与重复复核，让评审更好地融入开发者的反馈节奏。

03 token 成本

针对大量上下文读取与多轮验证，在效果约束下探索更优成本。

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造

专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践

专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业

专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering

专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理

专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付

专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software